Name: NETHRA CHANDRAGANDHI T **Roll No:** 240701357 **CSE**

Assignment 5 (Unit V): Overview and MCQs

Topic 1. The Limits of Low-Code Platforms

Overview

- **Vendor lock-in:** apps depend on proprietary runtime/metadata, so migration often means rebuilding.
- **Limited customization** of UI/logic, plus **scalability, performance and quota limits** (API calls, records, execution time).
- **Integration, security and compliance** gaps for legacy systems, data residency and fine-grained control.
- **Governance issues:** shadow IT, app sprawl, technical debt, weak testing/CI-CD, and rising licence cost at scale.
- **Poor fit:** real-time/high-performance systems or products where software is the core differentiator. **Fix:** fit assessment, CoE governance, open APIs, code extensibility.

Q1. Vendor lock-in means
a) Source code is encrypted **b)** The platform is free **c)** Difficulty moving apps to another platform **d)** User accounts get locked

Q2. A typical low-code limitation is
a) Unlimited customization **b)** No security needed **c)** Cannot build forms **d)** Constraints on deep customization

Q3. Shadow IT refers to
a) Night-shift IT work **b)** Apps built without IT's knowledge/approval **c)** Dark-mode UIs **d)** Backup servers

Q4. Least suited use case for low-code
a) Onboarding workflow **b)** Inventory tracker **c)** Real-time high-frequency trading engine **d)** Leave management app

Q5. API call limits are an example of
a) Normalization **b)** Platform quotas/constraints **c)** Design patterns **d)** Coding standards

Q6. Technical debt in low-code mostly comes from
a) Cloud hosting **b)** Version numbers **c)** Poorly designed, undocumented quick builds **d)** Too many developers

Q7. A governance measure for low-code is
a) Banning documentation **b)** Removing audits **c)** Disabling security **d)** Center of Excellence (CoE)

Q8. A cost concern at scale is
a) Zero maintenance **b)** Per-user/per-app licence growth **c)** No hosting **d)** Free licences

Q9. Legacy-system integration is hard because
a) Too many connectors **b)** Typing is slow **c)** Connectors for custom/legacy protocols may be missing **d)** Excess documentation

Q10. Multi-tenant architecture can cause
a) Dedicated hardware **b)** Zero latency **c)** Unlimited storage **d)** Shared-resource contention/performance limits

Q11. To reduce lock-in risk, prefer
a) Closed connectors **b)** Open APIs and data export **c)** Proprietary formats **d)** No backups

Q12. Compared to pro-code, debugging in low-code is often
a) Fully automatic **b)** More powerful **c)** Unnecessary **d)** Limited in depth and tooling

Q13. Security in low-code is
a) Not applicable **b)** Only the customer's **c)** A shared responsibility **d)** Only the vendor's

Q14. Data residency concerns
a) Data colour **b)** Number of forms **c)** Where data is physically stored **d)** Screen size

Q15. Which is NOT a typical limit?
a) Record/storage quotas **b)** Restricted UI customization **c)** Execution time limits **d)** Building forms visually

Q16. App sprawl is mainly a ____ problem
a) Governance **b)** Hardware **c)** Networking **d)** Typing

Q17. Low-code CI/CD and testing support is generally
a) Less mature **b)** Mandatory **c)** Always better than pro-code **d)** Impossible

Q18. If software is the core product, usually choose
a) Spreadsheets **b)** Custom pro-code development **c)** Pure low-code **d)** No development

Q19. Before adopting a platform you should
a) Fire IT **b)** Run a fit assessment / proof of concept **c)** Ignore compliance **d)** Skip evaluation

Q20. What helps overcome low-code limits?
a) Removing APIs **b)** Blocking integrations **c)** Disabling logs **d)** Code extensibility (scripts, functions, widgets)

Answer key: 1-c, 2-d, 3-b, 4-c, 5-b, 6-c, 7-d, 8-b, 9-c, 10-d, 11-b, 12-d, 13-c, 14-c, 15-d, 16-a, 17-a, 18-b, 19-b, 20-d

Topic 2. Glimpses of Pro-Code in Zoho Creator and Catalyst

Overview

- **Zoho Creator (low-code): Deluge** scripting for workflows, validations, schedules, custom functions and invokeUrl integrations; **Widgets** (HTML/CSS/JS), Plugins, Pages, REST API v2 with OAuth, and Connections.
- **Zoho Catalyst (pro-code, serverless):** functions in Java/Node.js/Python (Basic I/O, Advanced I/O, Event, Cron, Integration, Browser Logic).
- **Cloud Scale services:** Data Store (ZCQL), File Store, Cache, Authentication, Search, Mail. Also **Slate** (frontend hosting), **AppSail** (managed runtime), Circuits, Signals, QuickML/Zia, and Catalyst CLI/SDKs.
- **Typical pattern:** build UI and business app in Creator; call Catalyst functions for heavy logic, external SDKs and secure server-side code.

Q1. Deluge stands for
a) Data Enriched Language for the Understanding of Generated Events **b)** Data Entry Logic Utility **c)** Dynamic Engine Language for UI Generation **d)** Digital Event Logic Engine

Q2. Deluge is the scripting language of
a) Salesforce **b)** Zoho Creator **c)** Mendix **d)** OutSystems

Q3. Deluge task to call an external REST API
a) alert **b)** sendmail **c)** insert **d)** invokeUrl

Q4. Zoho Catalyst is
a) An email client **b)** A serverless full-stack platform **c)** A CRM **d)** A spreadsheet tool

Q5. Language NOT typically used in Catalyst functions
a) Node.js **b)** COBOL **c)** Java **d)** Python

Q6. Catalyst Data Store queries use
a) SQLite **b)** GraphQL **c)** HQL **d)** ZCQL

Q7. Catalyst function type that runs on a schedule
a) Cron function **b)** Event function **c)** Browser Logic **d)** Basic I/O

Q8. Creator Widgets are built using
a) HTML, CSS, JavaScript **b)** Only SQL **c)** Only Deluge **d)** Assembly

Q9. Catalyst component for hosting frontends
a) Circuits **b)** Cache **c)** Zia **d)** Slate

Q10. Connections in Creator are used to
a) Set up Wi-Fi **b)** Authenticate securely with external services **c)** Change page layouts **d)** Print reports

Q11. Catalyst service for file storage
a) Signals **b)** Data Store **c)** Cache **d)** File Store

Q12. Catalyst CLI is used for
a) Local development and deployment **b)** Only SMS **c)** Logo design **d)** Only backups

Q13. AppSail lets you
a) Design forms **b)** Create spreadsheets **c)** Deploy full apps on a managed runtime **d)** Write only Deluge

Q14. Why call Catalyst from Creator?
a) Change theme colour **b)** Add a form field **c)** Offload complex logic / use external SDKs **d)** Rename an app

Q15. Creator REST API authentication commonly uses
a) Telnet **b)** FTP **c)** OAuth 2.0 **d)** SMTP only

Q16. Serverless means developers
a) Write no code **b)** Cannot scale **c)** Don't manage server infrastructure **d)** Have no servers anywhere

Q17. Catalyst function type for HTTP/REST requests
a) Advanced I/O **b)** Event **c)** Cron **d)** Integration

Q18. Deluge custom functions are
a) Only visuals **b)** Hardware drivers **c)** Only reports **d)** Reusable blocks of logic

Q19. Catalyst Circuits is used for
a) Orchestrating serverless workflows **b)** Electrical wiring **c)** Image editing **d)** Email templates

Q20. Which is TRUE?
a) Creator is low-code; Catalyst is pro-code serverless **b)** Catalyst is no-code only **c)** Neither has APIs **d)** Creator is pro-code only

Answer key: 1-a, 2-b, 3-d, 4-b, 5-b, 6-d, 7-a, 8-a, 9-d, 10-b, 11-d, 12-a, 13-c, 14-c, 15-c, 16-c, 17-a, 18-d, 19-a, 20-a

Topic 3. The No-Code to Low-Code to Pro-Code Spectrum Revisited

Overview

- **No-code:** drag-and-drop for business users; fastest, least flexible (Airtable, Glide). **Low-code:** visual modelling plus scripting for citizen and professional developers (Zoho Creator, Power Apps, OutSystems). **Pro-code:** full programming by engineers; slowest but maximum control (Java, .NET, Zoho Catalyst).
- **Trade-off:** moving toward pro-code increases control, flexibility and effort; it reduces speed and accessibility.
- **Fusion teams** combine business users and developers. **Hybrid architecture:** low-code for UI/workflows, pro-code for complex or performance-critical logic (e.g., Creator + Catalyst).
- **Choose** by complexity, scalability, security, time-to-market, budget, skills and maintenance. Apps often evolve from no-code prototype to low-code app with pro-code components. AI is blurring the boundaries.

Q1. Fastest approach for simple apps
a) No-code **b)** Assembly **c)** Pro-code **d)** Hardware design

Q2. Highest flexibility and control
a) Spreadsheets **b)** Low-code **c)** Pro-code **d)** No-code

Q3. A citizen developer is
a) A DBA only **b)** A hacker **c)** A non-IT employee building apps with approved tools **d)** A government coder

Q4. A fusion team is
a) Only testers **b)** Nuclear engineers **c)** Business and IT collaborating on apps **d)** Only designers

Q5. Zoho Creator is an example of
a) Low-code **b)** No-code only **c)** An OS **d)** Pure pro-code

Q6. Zoho Catalyst is closer to
a) Spreadsheets **b)** No-code **c)** Presentation tools **d)** Pro-code

Q7. Moving from no-code to pro-code increases
a) Control and complexity **b)** Speed only **c)** Only drag-and-drop **d)** Nothing

Q8. A hybrid approach means
a) Using paper forms **b)** Combining low-code with custom code **c)** Avoiding code **d)** Using one tool only

Q9. Which should NOT drive the choice?
a) Colour of office walls **b)** Complexity **c)** Scalability **d)** Security needs

Q10. A typical no-code limitation
a) Limited extensibility **b)** Needs compilers **c)** Needs deep programming **d)** Unusable by business users

Q11. Pro-code development requires
a) Only templates **b)** Only mouse clicks **c)** No skills **d)** Professional programming skills

Q12. Low-code is
a) Never allows code **b)** Always slower than pro-code **c)** Platform-free **d)** Visual development with optional hand-coding

Q13. Rapid prototyping is most linked to
a) Hardware fabrication **b)** No-code / low-code **c)** Paperwork **d)** OS kernel writing

Q14. A performance-critical custom algorithm suits
a) A form builder **b)** A template **c)** No-code **d)** Pro-code

Q15. Typical app evolution path
a) Pro-code to no-code only **b)** Hardware to software **c)** No path exists **d)** No-code prototype, low-code app, pro-code parts

Q16. Correct pairing
a) Low-code: assembly **b)** Pro-code: no IDE **c)** Pro-code: drag-and-drop only **d)** No-code: business users

Q17. The spectrum view suggests
a) Only one valid method **b)** Three separate worlds **c)** Coding is obsolete **d)** A continuum with overlapping approaches

Q18. Benefit of Creator + Catalyst hybrid
a) Visual speed plus custom power **b)** Always slower **c)** No integration **d)** No flexibility

Q19. Governance matters most when
a) Citizen developers build many apps **b)** No one uses software **c)** Hardware is bought **d)** Only one app exists

Q20. What is blurring the three categories?
a) Typewriters **b)** Paper forms **c)** AI-assisted development **d)** Fax machines

Answer key: 1-a, 2-c, 3-c, 4-c, 5-a, 6-d, 7-a, 8-b, 9-a, 10-a, 11-d, 12-d, 13-b, 14-d, 15-d, 16-d, 17-d, 18-a, 19-a, 20-c

Topic 4. Career Pathways

Overview

- **Roles:** Citizen Developer, Low-Code Developer (e.g., Deluge), Solution Architect, Business/Process Analyst, Platform Administrator, Integration Specialist, Full-Stack/Pro-Code Developer, Automation/RPA Specialist, AI/Data Specialist, Consultant/Freelancer, CoE Lead.
- **Skills:** data modelling, SQL, REST APIs, JSON, JavaScript, Deluge/Python/Java, cloud and security basics; requirements analysis, UX, agile and testing; communication and collaboration; domain knowledge (finance, HR, healthcare, retail).
- **Certifications:** Zoho Creator, Microsoft Power Platform (PL-900, PL-100, PL-400), Salesforce, OutSystems, Mendix.
- **Getting started:** build a portfolio of real apps, join communities and hackathons, earn certifications, take internships/freelance work, and add pro-code skills to widen options.

Q1. A business user who builds apps with approved tools is a
a) Citizen developer **b)** Network engineer **c)** Kernel developer **d)** Game designer

Q2. Who designs end-to-end solutions and integrations?
a) Data entry operator **b)** Solution architect **c)** Receptionist **d)** Auditor only

Q3. Scripting language for a Zoho Creator developer
a) Fortran **b)** Pascal **c)** Deluge **d)** COBOL

Q4. PL-900 relates to
a) AWS storage **b)** Microsoft Power Platform **c)** Zoho Catalyst **d)** Oracle DB

Q5. A Platform Administrator manages
a) Novels **b)** UI icons **c)** Only sales **d)** Users, environments and security

Q6. Most useful skill for integration roles
a) Photography **b)** REST APIs and JSON **c)** Welding **d)** Calligraphy

Q7. A CoE lead focuses on
a) Only hardware **b)** Standards, training and governance **c)** Payroll only **d)** Only marketing

Q8. A good way to build a portfolio
a) Build and showcase real apps **b)** Avoid projects **c)** Copy without learning **d)** Hide your work

Q9. A Business Analyst typically
a) Installs OS only **b)** Gathers requirements and maps processes **c)** Designs chips **d)** Repairs hardware

Q10. Learning pro-code alongside low-code
a) Makes you obsolete **b)** Widens career opportunities **c)** Removes options **d)** Blocks promotions

Q11. NOT a typical low-code role
a) Platform admin **b)** Automation specialist **c)** Blacksmith **d)** Low-code developer

Q12. Freelancing in low-code involves
a) Avoiding clients **b)** Only teaching **c)** Delivering apps to clients per project **d)** Only full-time jobs

Q13. Most valuable soft skill for fusion teams
a) Silence **b)** Communication and collaboration **c)** Working in isolation **d)** Avoiding meetings

Q14. Data modelling helps a developer to
a) Avoid testing **b)** Type faster **c)** Paint walls **d)** Design efficient forms and relationships

Q15. An RPA/Automation specialist builds
a) Hardware parts **b)** Furniture **c)** Automated workflows and bots **d)** Paper forms

Q16. Domain knowledge is valuable because
a) Apps can fit real business needs **b)** It is irrelevant **c)** It replaces coding **d)** It removes testing

Q17. Which activity helps career growth?
a) Ignoring updates **b)** Skipping learning **c)** Hackathons and forums **d)** Avoiding networking

Q18. A full-stack developer in this ecosystem might
a) Avoid APIs **b)** Only design posters **c)** Manage payroll **d)** Build Catalyst services to extend apps

Q19. Certifications mainly help to
a) Avoid learning **b)** Validate skills to employers **c)** Block job offers **d)** Replace all experience

Q20. Strongest long-term combination
a) One tool, no updates **b)** Low-code + pro-code + domain knowledge **c)** No technical skills **d)** Memorizing menus

Answer key: 1-a, 2-b, 3-c, 4-b, 5-d, 6-b, 7-b, 8-a, 9-b, 10-b, 11-c, 12-c, 13-b, 14-d, 15-c, 16-a, 17-c, 18-d, 19-b, 20-b

Topic 5. The Future of Low-Code

Overview

- **AI-assisted development:** generative AI and copilots (Microsoft Copilot in Power Apps, Zoho Zia) build apps, formulas and workflows from natural language; **agentic AI** executes multi-step tasks.
- **Hyperautomation** (low-code + RPA + AI + process mining + integration), **composable enterprise**, **multi-experience** apps (web, mobile, chat, voice), industry-specific solutions, IoT/edge and analytics integration.
- **Pro-code convergence:** Git, CI/CD, APIs and stronger extensibility; **open standards** to reduce lock-in; stronger governance and security.
- **Challenges:** quality/security of AI-generated apps, sprawl, skills, privacy. **Outlook:** low-code will not replace developers; their focus shifts to architecture, integration and governance.

Q1. Technology most transforming low-code today
a) Generative AI **b)** Floppy disks **c)** Dial-up modems **d)** Punch cards

Q2. Hyperautomation combines
a) Only spreadsheets **b)** Only manual work **c)** Only hardware **d)** RPA, AI, low-code and integration

Q3. Zoho's AI assistant is
a) Alexa **b)** Zia **c)** Siri **d)** Cortana

Q4. Microsoft's AI assistant in Power Apps
a) Bixby **b)** Zia **c)** Copilot **d)** Einstein

Q5. Composable enterprise emphasizes
a) Paper processes **b)** Monolithic systems **c)** Reusable building blocks assembled quickly **d)** No reuse

Q6. Trend linking low-code to DevOps
a) Manual deployment only **b)** Git integration and CI/CD **c)** Removing version control **d)** Banning APIs

Q7. Agentic AI refers to
a) A printer driver **b)** A database index **c)** AI planning and doing multi-step tasks autonomously **d)** A travel agency

Q8. Multi-experience apps target
a) Only print **b)** Only desktop **c)** Only email **d)** Web, mobile, chat and voice from one design

Q9. A key future challenge
a) Governance and quality of AI-generated apps **b)** Excess dial-up **c)** Typewriter supply **d)** Too few computers

Q10. Will low-code fully replace developers?
a) Yes, entirely **b)** Only in banking **c)** No; their role shifts to complex/architectural work **d)** Yes, within a year

Q11. Gartner forecasts suggest low-code will
a) Vanish **b)** Leave only no-code **c)** Ban pro-code **d)** Account for a large share of new apps

Q12. Democratization of development means
a) Only experts build **b)** More people can build apps **c)** Banning citizen developers **d)** Voting for developers

Q13. Natural-language app generation means
a) Using Morse code **b)** Drawing by hand only **c)** Describing an app in plain language to build it **d)** Typing in binary

Q14. What reduces lock-in in future?
a) Proprietary-only APIs **b)** Open standards and portability **c)** No data export **d)** Closed formats

Q15. Process mining is used to
a) Analyze real business processes from data **b)** Clean offices **c)** Mine metals **d)** Mine only crypto

Q16. Vertical/industry solutions provide
a) Only game engines **b)** Hardware **c)** Prebuilt templates for specific sectors **d)** Nothing generic

Q17. Future developer roles focus more on
a) Architecture, integration and governance **b)** Typing speed **c)** Manual data entry **d)** Printing documents

Q18. A risk of AI-generated apps
a) Always cheaper **b)** Security flaws and quality issues **c)** No risk **d)** Guaranteed perfection

Q19. IoT and edge integration shows low-code
a) Paper-only **b)** Has no future **c)** Shrinking **d)** Expanding beyond simple business apps

Q20. Best summary of low-code's future
a) Fully obsolete **b)** Limited to spreadsheets **c)** Developer-free **d)** AI-enhanced, governed, integrated with pro-code

Answer key: 1-a, 2-d, 3-b, 4-c, 5-c, 6-b, 7-c, 8-d, 9-a, 10-c, 11-d, 12-b, 13-c, 14-b, 15-a, 16-c, 17-a, 18-b, 19-d, 20-d